Let's review what we covered:

1. We saw how to name and version our images by giving them tags, either after they're built:

   ```
   $ docker tag a1df0eddba18 railsapp
   ```

   or at build time (here setting two tags):

   ```
   $ docker build -t railsapp -t railsapp:1.0 .
   ```

2. We added a default command to our image using the CMD instruction:

   ```
   CMD ["bin/rails", "s", "-b", "0.0.0.0"]
   ```

3. We sped up our image builds by using a .dockerignore to prevent unnecessary files from being sent to the Docker daemon as part of our build context.

4. We ensured that we always use up-to-date package repository information when altering the packages we install by combining apt-get update and apt-get install into a single RUN instruction:

   ```
   RUN apt-get update -yqq && apt-get install -yqq --no-install-recommends \
     nodejs
   ```

5. We prevented file changes from causing our gems to be rebuilt by copying our Gemfiles earlier in our Dockerfile so they could be cached separately:

   ```
   COPY Gemfile* /usr/src/app/
   WORKDIR /usr/src/app
   RUN bundle install
   ```

6. Finally, we indicated who was responsible for our image by setting a maintainer with the LABEL instruction:

   ```
   LABEL maintainer="rob@DockerForRailsDevelopers.com"
   ```

Not bad for a day's work.

If you thought using Docker for development couldn't get any better, or perhaps still have some reservations, brace yourselves. Up next, we'll discover an even more powerful tool that will supercharge our development. Onward!